



UNIVERSIDAD AUTÓNOMA
DE ZACATECAS

"FRANCISCO GARCÍA SALINAS"



UNIDAD ACADÉMICA DE INGENIERÍA ELÉCTRICA
INGENIERÍA DE SOFTWARE 4° "D"

Proyecto: FashionFlow POS

Materia: Análisis y Diseño Orientado a Objetos

Docente: Cristian Boyain

Integrante 1: Julio Miguel Sifuentes Sánchez

Integrante 2: Ana Patricia Galvan Cruz

Versión: 3.0

Fecha: 21 de mayo de 2026

FashionFlow POS | SRS v3.0 | Análisis y Diseño Orientado a Objetos | Mayo 2026

5. Diseño Orientado a Objetos

5.1 Jerarquía de Herencia — Productos

La clase abstracta **Producto** centraliza los atributos y métodos comunes. Cada departamento es una subclase que hereda de ella y puede extender sus propiedades según sus necesidades específicas.

sku : String
nombre : String
precio : Float
existencias : Int
departamento : String
obtener_info() : String
aplicar_descuento(p) : Float

Azul = atributos | Verde = métodos

Las siguientes clases heredan de **Producto**:

Calzado	talla : String material : String	CalzadoDama CalzadoCaballero	Hereda de Producto
CalzadoDama	tacon : Bool	—	Hereda de Calzado
CalzadoCaballero	tipo : String	—	Hereda de Calzado
RopaDeportiva	deporte : String genero : String	—	Hereda de Producto
VestidoDeNoche	ocasion : String tela : String	—	Hereda de Producto
RopaHombre	talla : String	—	Hereda de Producto
RopaMujer	talla : String	—	Hereda de Producto
Accesorio	tipo_accesorio : String	—	Hereda de Producto

5.2 Diagrama de Clases Principal

Principales clases del sistema y sus relaciones:

Producto	Inventario	Venta	DetalleVenta	o	Usuario	Reporte
sku	productos[]	id	producto	nombre	id	tipo

nombre	umbral	fecha	cantidad	productos[]	usuario	rango
precio		cajero	precio		rol	
stock		total				
info()	agregar()	calcular()	subtotal()	agregar()	autenticar()	generar()
descuento()	reducir()	cerrar()		listar()		exportar()
	buscar()					

Azul = atributos | Verde = métodos

6. Casos de Uso

6.1 Actores

Cajero	Primario	Realiza ventas y consulta inventario
Administrador	Primario	Gestiona el sistema completo
Sistema	Secundario	Ejecuta procesos automáticos (alertas, cálculos)

6.2 Diagrama de Casos de Uso

CU-01 Iniciar Sesión	CU-04 Agregar / Editar Producto
CU-02 Registrar Venta	CU-05 Generar Reporte
CU-03 Consultar Inventario	CU-06 Gestionar Departamentos

Azul = Cajero y Admin | Morado = solo Administrador

6.3 Especificación de Casos de Uso

CU-01	Iniciar Sesión	Cajero / Admin	Ingresa credenciales → sistema valida y otorga acceso según rol.
CU-02	Registrar Venta	Cajero	Busca producto → agrega al carrito → calcula total → registra transacción y actualiza inventario.
CU-03	Consultar Inventario	Cajero / Admin	Filtra por departamento o búsqueda → visualiza existencias.
CU-04	Agregar / Editar Producto	Administrador	Ingresa o modifica datos del producto y lo asigna a un departamento.
CU-05	Generar Reporte	Administrador	Selecciona tipo y rango de fechas → sistema genera y permite exportar.
CU-06	Gestionar Departamentos	Administrador	Crea o renombra departamento → sistema instancia el módulo sin alterar clases existentes.

7. Arquitectura del Sistema — Patrón MVC

El sistema utiliza el patrón **Modelo-Vista-Controlador (MVC)** para separar responsabilidades y facilitar el

mantenimiento. Además se organiza en módulos independientes: Inventario, Ventas, Reportes y Login.

Administra datos y lógica del sistema	Muestra la interfaz gráfica al usuario	Conecta Vista y Modelo; procesa las acciones
Productos, Inventario, Usuarios, Reportes	Pantalla de Login, Ventana de Ventas, Ventana de Inventario, Ventana de Reportes	Registrar venta, Actualizar inventario, Validar usuarios, Generar reportes

Módulos del Sistema

Inventario	Controla existencias y productos	Altas, bajas y modificaciones de productos
Ventas	Gestiona operaciones de venta	Tickets, cálculos y registros de transacciones
Reportes	Genera información del negocio	Reportes de ventas e inventario por departamento
Login	Controla acceso y seguridad	Inicio de sesión y validación de usuarios por rol

Principios SOLID Aplicados

S — Responsabilidad Única	Cada clase tiene una sola razón de cambio; por ejemplo, la clase Venta solo gestiona transacciones.
O — Abierto / Cerrado	Nuevos departamentos se agregan como subclases sin modificar la clase Producto .
L — Sustitución de Liskov	Cualquier subclase de Producto puede usarse donde se espera un Producto.
D — Inversión de Dependencias	Los controladores dependen de abstracciones, no de implementaciones concretas.

8. Conclusión

FashionFlow POS es una propuesta de sistema enfocada en mejorar la administración de tiendas de ropa y calzado.

Durante el desarrollo se aplicarán conceptos de programación orientada a objetos, herencia y modularidad para crear un sistema organizado y funcional.

La arquitectura MVC facilitará mantener una mejor estructura en el programa y simplificará futuras mejoras. La jerarquía de clases permite gestionar de forma natural la diversidad de productos, mientras que los módulos

independientes permiten detectar errores y agregar funciones sin afectar el resto del sistema.

Este documento servirá como base para las siguientes etapas del proyecto: diseño de interfaces, programación y pruebas.